PACE: Viva Guide

Carbon- and SLA-aware edge–cloud scheduling with deep reinforcement learning · 22AIE401 Reinforcement Learning

Nandu Manoj · Naren Sundar L · Vatturu Pardheev · Charuvarthan T · Guide: Dr. Palmani Duraisamy

THE PROJECT IN 30 SECONDS

Every job (for example, a camera alert) is a small graph of tasks. For each task, we have to choose **where** to run it: on the camera box itself (edge), on a nearby server in Mumbai (fog), or on a far server in Stockholm (cloud). Mumbai is close but runs on **dirty** coal power; Stockholm is far but runs on **clean** power. So every choice trades **meeting the deadline** against **carbon emitted**. We trained an RL agent, **PACE**, that makes this choice for every task in about 1 ms. It beat the base paper (MERSEM) in simulation and on real AWS machines.

The setting (what the agent controls)

Edge · Coimbatore
camera boxes (our laptop)
slow (1 core) · no network delay
grid ≈ 560 gCO_2/kWh

Fog · AWS Mumbai
2 servers (x86 + ARM)
near (~30–110 ms)
grid ≈ 680 gCO_2/kWh
(dirty)

Cloud · AWS Stockholm
2 servers (x86 + ARM)
far (~150–400 ms)
grid ≈ 35 gCO_2/kWh
(clean)

Three kinds of jobs

Class	Example	Deadline	Notes
A · Alert	camera frame → detect pedestrian → alert	1 second	≈ 98% of all jobs; 30% of cameras are private (must stay in India)
B · Analytics	graph job from the dataset (many tasks)	seconds	big DAGs, hardest to finish in time
C · Batch	graph job from the dataset	30–120 min	can be **deferred** to a greener hour

Words you will hear

SLA / deadline miss	A job finished after its deadline. Our main metric: % of class A+B jobs that missed.
DAG	Directed acyclic graph: a job made of tasks, where a task can start only after its parent tasks finish.
Dynamic carbon	The CO_2 caused by running our tasks (the part a scheduler controls), in grams.
w (preference)	A number from 0 to 1. **w = 1**: care only about deadlines. **w = 0**: care only about carbon. It is an *input* to our policy.
EFT / CA-EFT	Simple rules. EFT = "run where it finishes earliest". CA-EFT = "run on the greenest node that still meets the deadline". CA-EFT is our strongest simple baseline.
Digital twin	Our simulator, tuned with measurements from the real AWS machines, so it behaves like the real testbed.

1. Existing work vs our work

The base paper: MERSEM (Ramicetty et al., arXiv 2026)

Problem they solve: the same one as ours: schedule graph-analytics jobs on edge, fog and cloud so that few jobs miss deadlines and little carbon is emitted.

What they do, intuitively: every **15 minutes**, they **predict** which jobs will arrive in the next 15 minutes, then run an **evolutionary (genetic) algorithm** that searches for a good "job → machine" plan. The RL part is small: a **tabular Q-learning** agent only decides *how strongly to mutate* the plans during the search (5 choices). Their objective is a fixed weighted sum: $score = w_s \cdot (SLA\ miss\ rate) + w_c \cdot (carbon)$, with $w_s + w_c = 1$.

Dataset they use: the **Rezaee & Adabi DAG workflow dataset** (Zenodo, CC-BY-4.0). We inspected it: **51,895 jobs** and **1.3 million tasks**, typically 7–62 tasks per job, with task sizes, dependencies and job deadlines. **We use the same dataset** (first 70% of the timeline for training, last 30% for testing).

How they evaluate: simulation only (an extended EdgeCloudSim), against other metaheuristics (PSO-GA, GWO, ...).

Side by side

	MERSEM (base paper)	PACE (ours)
When it decides	Once per 15-min epoch, offline, from a prediction	**Online**: the moment each task is ready (≈ 1 ms)
What it places	A whole job on one machine	**Each task** separately (a job can span edge + cloud)
Role of RL	Q-learning tunes the mutation strength of a genetic search	**The RL agent is the scheduler** (deep RL, PPO)
Deadline vs carbon	Fixed weight; re-run the search to change it	**One policy for every w**; a controller changes w live
Knowledge assumed	Exact task runtimes	Only **noisy estimates** (like reality)
Failures, priorities	Not modelled	Crashed nodes masked, tasks re-placed; alert > analytics > batch
Safety	None	Safety shield + privacy rules
Baselines	Other metaheuristics only	EFT, CA-EFT (strong simple rule), and **MERSEM itself**
Evaluation	Simulation only	Simulation **+ real AWS testbed** (Coimbatore, Mumbai, Stockholm)

What we took from others vs what we built

TAKEN (CITE THESE)

- Problem, metrics, dataset: **MERSEM**
- Online RL scheduling, BC warm start: **DeepRM** (2016, 2018)
- One shared network that scores each node: **Decima** (2019)
- **PPO** (Schulman 2017); preference-conditioned RL (2019)
- Controller on a weight: PID-Lagrangian safe RL (2020)

BUILT BY US

- **PACE** agent, and the **deadline-decomposition reward**
- Job-pressure features, safety shield, SLA controller
- Our **re-implementation of MERSEM** (no public code)
- Simulator + **real AWS testbed** + dashboard
- Three **sim-to-real** fixes; all experiments E1–E9

2. Our approach: PACE as an RL problem

THE RL LOOP IN ONE PICTURE

Environment: simulator during training · real AWS testbed when deployed

↓ a task becomes ready

State s: facts about the task + facts about every candidate node

↓

Agent (policy network): gives a probability to each node, plus "defer"

↓

Action a: run the task on node *m* (or defer a batch task)

↓ task runs, job finishes later

Reward r: minus (carbon used + penalty if late) → agent learns from it

State (what the agent sees)

- **Task:** its class, size, how much time is left (budget), how much work remains after it, whether it is private, and the weight **w**.
- **Each node:** tier, speed, estimated wait, transfer and finish time, grid carbon now and forecast, the task's estimated carbon there, load.

Action (what it can do)

- Pick one node, or **Defer** (only batch jobs with spare time).
- **Masks** remove illegal choices: crashed nodes, private data going abroad, other cameras' boxes.

Reward (what it is paid for)

- Lose points for **carbon** (scaled by 1 − w).
- Lose points for **lateness** (scaled by w): half for the task's own **sub-deadline**, half for the whole job's deadline.

KEY IDEA: DEADLINE DECOMPOSITION

A big job has dozens of tasks. If we only punish the job at the end, the agent cannot tell *which* task caused the miss. So each task gets its own sub-deadline = job deadline − time still needed after it. Now each decision is judged fairly. Before this fix, the agent ignored w and missed 45% of analytics deadlines.

The policy network (why it works for any number of nodes)

Input per node node facts + task facts (incl. w) → **Same small MLP** for every node (2 × 128 units) → **One score per node** + a score for "defer" → **Masked softmax** probabilities over legal actions

Because the **same** network scores every node, it works with 6 nodes or 30, and a crashed node is simply masked out. It has about 60,000 parameters and decides in about 0.55 ms. A separate **critic** estimates how good the state is (needed for PPO).

At deployment: two extra safety layers

- **SLA-target controller:** the operator says "at most 5% missed". If misses go up, the controller raises w; if there is headroom, it lowers w to save carbon. **No retraining**, because w is just an input.
- **Safety shield:** before the agent chooses, remove nodes that clearly cannot finish the task in time.

3. The important formulas, explained simply

① Return: what the agent maximises

$$G = r_1 + \gamma \cdot r_2 + \gamma^2 \cdot r_3 + \ldots \quad (\gamma = 0.8)$$

Total reward from now on, with future rewards counted a bit less. γ = 0.8 is small on purpose: a placement mostly affects its own job.

② Our reward for placing task i

$$r = -\left[(1 - w) \cdot \text{carbon}_i + w \cdot \left(\tfrac{1}{2} \cdot \text{late}_{\text{task}} + \tfrac{1}{2} \cdot \text{late}_{\text{job}} \right) \right]$$

carbon_i is the task's CO_2, normalised so both parts have similar size. late = 0 if on time, otherwise 1 + how late (capped at 2). w = 1: only deadlines matter; w = 0: only carbon matters.

③ Policy: probability of choosing node m

$$\pi(m \mid s, w) = e^{z_m} / \Sigma_{\text{legal } k}\, e^{z_k}$$

Softmax over the scores z of the legal nodes only (masked). At deployment we simply take the node with the highest probability.

④ Advantage: was this action better than usual?

$$A = (\text{return actually obtained}) - V(s)$$

V(s) is the critic's guess of the return from this state. A > 0: the action was better than expected, so make it more likely. We use GAE (λ = 0.9), a smoothed version of this.

⑤ PPO update: improve, but not too much at once

$$\text{ratio} = \pi_{\text{new}}(a|s) \,/\, \pi_{\text{old}}(a|s) \qquad L_{\text{PPO}} = -\min\left(\text{ratio} \cdot A \,,\, \text{clip}(\text{ratio}, 0.8, 1.2) \cdot A \right)$$

Increase the probability of good actions (A > 0) and decrease bad ones, but the clip stops the policy changing by more than ±20% per update. This keeps training stable.

⑥ Full training loss

$$L = L_{\text{PPO}} + 0.5 \cdot (V(s) - G)^2 - \beta \cdot \text{entropy}$$

Policy part + critic error (learn V) − an entropy bonus that keeps the agent exploring early on (β goes from 0.01 down to 0.002).

⑦ Behaviour cloning (warm start before PPO)

$$L_{\text{BC}} = -\log \pi(\text{ action CA-EFT would take} \mid s)$$

First copy a good simple rule, so PPO does not start from random placements that miss almost every deadline.

⑧ SLA controller (sets w live)

$$e = \text{miss rate} - \text{target}$$
$$w = w_0 + 0.8 \cdot e + 0.08 \cdot \Sigma e$$

Too many misses → e > 0 → w goes up. A "PI controller", like a thermostat.

⑨ Safety shield

$$\text{keep node if: time to finish} \le (1 - 0.6w) \cdot \text{budget}$$

At w = 1, a node must finish within 40% of the remaining time to be allowed.

⑩ Carbon of a task

$$CO_2 = \text{extra power} \times \text{run time} \times \text{grid intensity}$$

Grid intensity (g/kWh) is what makes Mumbai ≈ 20× dirtier than Stockholm.

⑪ SLA miss rate (main metric)

$$\text{miss rate} = (\#\ \text{A+B jobs late}) \,/\, (\#\ \text{A+B jobs})$$

The same metric MERSEM uses.

4. Step by step: how we built, trained and use PACE

Training flow (offline, on the laptop)

1 Data dataset jobs (train split) + 3 job classes → **2 Simulator** random worlds: nodes, load, network, noise → **3 Copy CA-EFT** behaviour cloning, 3 rounds → **4 PPO** 82 rounds, w random each episode (≈35 min) → **5 pace.pt** final policy

- **Step 2, domain randomisation:** every training episode is a different world (2–6 edge, 1–3 fog, 1–3 cloud nodes; network ×0.5–1.5; noisy runtime estimates). This is why the policy copes with situations it has not seen.
- **Step 4:** each episode picks a random w, so **one network learns every trade-off** (preference-conditioned RL). ≈ 3.5 million decisions in total.

From simulation to the real machines

6 Calibrate measure real speeds, delays, contention → **7 Digital twin** simulator with real numbers → **8 Fine-tune** PPO again in the twin → pace_live.pt → **9 Real runs** compare with MERSEM, CA-EFT

THREE SIM-TO-REAL GAPS WE FOUND AND FIXED (GOOD VIVA STORY)

1. Dispatch: the orchestrator held a CPU slot during each network call → now tasks are sent as soon as they are placed. **2. CPU contention:** a 2-vCPU x86 server is really 1 physical core; two tasks at once ran 1.7–1.8× slower → now measured per node. **3. Network drift:** the link to Mumbai got slower in the evening, so alerts sent there were late → we added an **online network estimate** (add the recent 90th-percentile extra delay to every estimate).

Usage flow (what happens live, for every task)

Task ready its parent tasks finished → **Build state** estimates + live network correction → **Mask + shield** remove illegal / too-slow nodes → **Policy picks** node with highest probability → **Run + learn w** result returns; controller updates w

What we ran (the experiments)

ID	Where	Question it answers
E1	Simulation	Normal load: who misses fewest deadlines, with how much carbon? (10 seeds)
E2–E5	Simulation	Stress: more load, wrong runtime estimates, node crashes, rush hour
E6	Simulation	Setups never seen in training: bigger city, new region, heavier jobs, slow network
E8 / E8b	Real AWS	Same comparison on real machines (E8b = with the network fix), 6 paired runs
E9	Both	How close was the simulator's prediction to the real result?

Fair comparison: every method sees the same jobs and carbon trace in each seed; tests use only held-out jobs; we report paired Wilcoxon tests.

5. Results, positives, and viva answers

Experiment	MERSEM	CA-EFT	PACE	Meaning
Simulation, normal load: deadlines missed	13.3%	2.8%	**1.3%**	10× fewer misses than the base paper
… dynamic carbon (g)	0.51	0.74	**0.39**	−22% vs MERSEM, −47% vs CA-EFT
2× load: missed	26.3%	7.2%	**3.0%**	MERSEM's 15-min plan goes stale
Unseen: 2× larger city: missed	17.3%	4.6%	**0.4%**	shared node scorer generalises
Unseen: much slower network: missed	24.8%	**1.4%**	17.3%	**our failure case**
Real AWS, normal / busy: missed	9.4 / 14.3%	3.7 / 6.2%	3.8 / 6.6%	beat MERSEM in 6/6 runs ($p = 0.031$)

RL ablation: behaviour cloning alone just copies CA-EFT (2.8%, 0.78 g) and ignores w. PPO is what makes w work: from $w = 0$ to 0.9, misses fall 11.5% → 1.3% while carbon rises only 0.30 → 0.39 g.

POSITIVES

- Far fewer deadline misses than the base paper, with less carbon (simulation)
- Fast: ≈ 1 ms per decision, reacts instantly to crashes and rush hours
- One network for every trade-off; the SLA target can change live
- Works on bigger networks and a new region without retraining
- Tested on real machines, not just simulation

NEGATIVES (SAY THESE BEFORE THEY ASK)

- Large analytics jobs: plain PACE misses 23% vs CA-EFT's 6% (the shield fixes most of it)
- Very slow network outside training: 17% vs CA-EFT's 1.4%
- On real AWS, PACE used 15% more carbon than MERSEM (not significant)
- Carbon data is synthetic (built from real yearly averages)
- Small testbed: 6 nodes, 3 seeds per load

Likely viva questions, short answers

Q: Why RL and not a simple rule? The best choice depends on many changing things (queues, carbon, network, DAG). A rule like CA-EFT must be re-tuned when goals change; our one policy covers every w and still uses 15–47% less carbon than CA-EFT.

Q: Why PPO? Stable (clipped updates), works with large discrete action sets and masks, and is the standard choice for policy-gradient training in simulators.

Q: Why behaviour cloning first? A random policy misses almost every deadline, so it learns slowly. Copying CA-EFT gives a sensible start; PPO then improves on its teacher.

Q: What is new compared with MERSEM? Online task-level decisions, deep RL as the scheduler, one policy for all w plus a live controller, failure handling, and real-machine validation.

Q: Is comparing with MERSEM fair? We were generous: our MERSEM sees the *actual* next-epoch jobs (perfect prediction) and falls back to EFT on failures. Everyone gets the same jobs and carbon traces.

Q: Why does PACE fail on the slow network? That network is far outside what it saw in training; CA-EFT simply recomputes finish times. Fix: wider network randomisation in training plus the online network estimate.

Q: How do you measure carbon on AWS? AWS gives no power meter, so: CPU time × published watts per vCPU × PUE × the grid's carbon intensity (the same model as MERSEM).